

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 5

Comprehensive Study of CNN Training, Regularization, Optimization, Hyperparameter Tuning, Transfer Learning and Cross-Validation

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

GitHub Repository:

<https://github.com/Snigdha-clo/Deep-Learning/tree/main/Comprehensive%20Study%20of%20CNN>

1 Objective

1. Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance.

The experiment uses a single CNN architecture, MobileNetV2, and the Oxford-IIIT Pet dataset. Students will analyze the effect of individual design choices using training/validation curves and finally select a suitable configuration using 5-fold cross-validation.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain different weight initialization strategies;
- identify and analyze overfitting using training and validation curves;
- compare SGD, Momentum, RMSProp and Adam;
- explain CNN hyperparameters such as kernel size, stride, padding and pooling;
- understand Batch Normalization and Dropout;
- perform transfer learning and fine-tuning using MobileNetV2;
- tune important training hyperparameters;
- apply 5-fold cross-validation for model selection; and
- justify the final model using accuracy, variability and computational cost.

3. Dataset and Experimental Setup

Dataset: Oxford-IIIT Pet Dataset

The Oxford-IIIT Pet Dataset contains images of cats and dogs belonging to 37 breeds. The images are RGB and have different spatial dimensions. For this experiment, all images are resized to

$$224 \times 224 \times 3.$$

The dataset is suitable for demonstrating transfer learning using ImageNet-pretrained CNNs.

Experimental considerations:

- Use RGB images resized to 224×224 .
- Normalize the input images according to the pretrained model requirements.
- Maintain separate training, validation and test data.
- The test set must remain untouched during hyperparameter selection.
- Use MobileNetV2 because it is computationally lighter than many large CNN architectures and is more suitable for CPU-based laboratory work.

Output

```
Downloading data from https://www.robots.ox.ac.uk/~vgg/data/pets/data/images.tar.gz
791918971/791918971 41s 0us/step
Extracting images...
Total images found: 7390
Number of classes: 37
Train: 5173, Val: 1108, Test: 1109
Train batches: 162
Val batches: 35
Test batches: 35
```

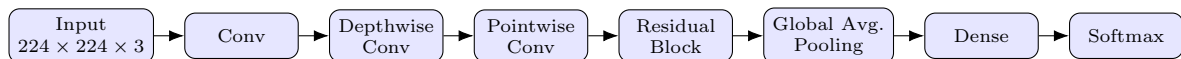
4. MobileNetV2 Architecture

MobileNetV2 is a lightweight CNN architecture designed to reduce computational complexity while maintaining good image representation.

Its important components include:

- depthwise convolution;
- pointwise 1×1 convolution;
- inverted residual blocks;
- linear bottlenecks;
- batch normalization; and
- ReLU6 activation.

A simplified representation is:



Note: The above diagram is a simplified conceptual representation rather than the complete MobileNetV2 architecture.

5. Weight Initialization

Weight initialization determines the initial values of trainable weights before training begins. A suitable initialization can improve convergence and reduce problems such as vanishing or exploding activations.

Students should investigate:

- Zero initialization
- Random initialization
- Xavier/Glorot initialization
- He initialization

Expected Analysis

Students should compare the initialization strategies using:

Plot 1: Training Loss vs. Epoch

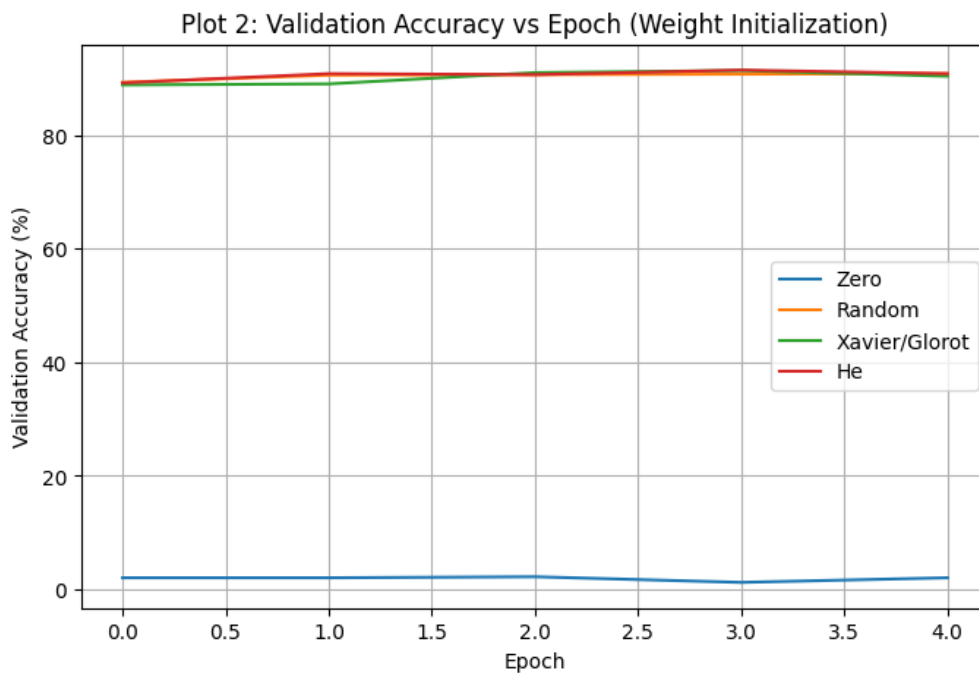
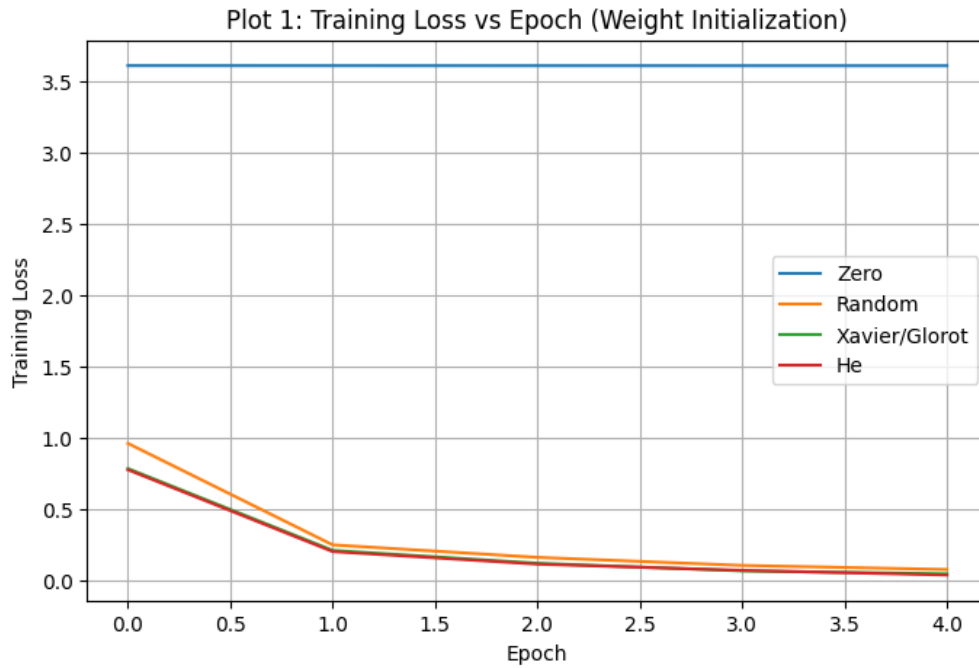
- X-axis: Epoch
- Y-axis: Training Loss
- Plot one curve for each initialization method.

Plot 2: Validation Accuracy vs. Epoch

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Plot one curve for each initialization method.

Students should identify which initialization gives faster and more stable convergence.

Output



--- Training with Zero initialization ---

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet_v2/mobilenet_v2_weights_9406464/9406464 2s 0us/step

Epoch 1/5

162/162 65s 287ms/step - accuracy: 0.0267 - loss: 3.6113 - val_accuracy: 0.0208 - val_loss: 3.6119

Epoch 2/5

162/162 15s 94ms/step - accuracy: 0.0284 - loss: 3.6109 - val_accuracy: 0.0208 - val_loss: 3.6127

Epoch 3/5

162/162 15s 93ms/step - accuracy: 0.0275 - loss: 3.6108 - val_accuracy: 0.0226 - val_loss: 3.6136

Epoch 4/5

162/162 21s 95ms/step - accuracy: 0.0278 - loss: 3.6106 - val_accuracy: 0.0126 - val_loss: 3.6141

Epoch 5/5

162/162 20s 94ms/step - accuracy: 0.0298 - loss: 3.6105 - val_accuracy: 0.0208 - val_loss: 3.6148

--- Training with Random initialization ---

Epoch 1/5

162/162 36s 171ms/step - accuracy: 0.7703 - loss: 0.9583 - val_accuracy: 0.8926 - val_loss: 0.3685

```

Epoch 2/5
162/162 15s 92ms/step - accuracy: 0.9217 - loss: 0.2453 - val_accuracy: 0.9052 - val_loss: 0.3043
Epoch 3/5
162/162 18s 112ms/step - accuracy: 0.9517 - loss: 0.1576 - val_accuracy: 0.9061 - val_loss: 0.2852
Epoch 4/5
162/162 15s 94ms/step - accuracy: 0.9725 - loss: 0.1011 - val_accuracy: 0.9070 - val_loss: 0.2912
Epoch 5/5
162/162 21s 94ms/step - accuracy: 0.9822 - loss: 0.0725 - val_accuracy: 0.9079 - val_loss: 0.2862

--- Training with Xavier/Glorot initialization ---
Epoch 1/5
162/162 36s 170ms/step - accuracy: 0.7920 - loss: 0.7811 - val_accuracy: 0.8881 - val_loss: 0.3529
Epoch 2/5
162/162 17s 104ms/step - accuracy: 0.9329 - loss: 0.2066 - val_accuracy: 0.8899 - val_loss: 0.3510
Epoch 3/5
162/162 15s 92ms/step - accuracy: 0.9662 - loss: 0.1165 - val_accuracy: 0.9097 - val_loss: 0.2952
Epoch 4/5
162/162 21s 94ms/step - accuracy: 0.9863 - loss: 0.0628 - val_accuracy: 0.9134 - val_loss: 0.3063
Epoch 5/5
162/162 16s 100ms/step - accuracy: 0.9909 - loss: 0.0420 - val_accuracy: 0.9034 - val_loss: 0.3160

--- Training with He initialization ---
Epoch 1/5
162/162 36s 173ms/step - accuracy: 0.7903 - loss: 0.7728 - val_accuracy: 0.8917 - val_loss: 0.3527
Epoch 2/5
162/162 16s 98ms/step - accuracy: 0.9362 - loss: 0.1983 - val_accuracy: 0.9079 - val_loss: 0.3034
Epoch 3/5
162/162 15s 92ms/step - accuracy: 0.9673 - loss: 0.1102 - val_accuracy: 0.9061 - val_loss: 0.3056
Epoch 4/5
162/162 15s 93ms/step - accuracy: 0.9820 - loss: 0.0662 - val_accuracy: 0.9143 - val_loss: 0.2981
Epoch 5/5
162/162 15s 94ms/step - accuracy: 0.9954 - loss: 0.0338 - val_accuracy: 0.9079 - val_loss: 0.3027

```

6. Regularization and Overfitting

Overfitting occurs when a model performs very well on the training data but generalizes poorly to unseen data. Students should compare:

- No regularization
- L2 regularization
- Dropout
- Batch Normalization

Expected Plots

Plot 3: Training and Validation Accuracy vs. Epoch

- X-axis: Epoch
- Y-axis: Accuracy (%)
- Plot separate curves for training and validation accuracy.

Students should identify the generalization gap.

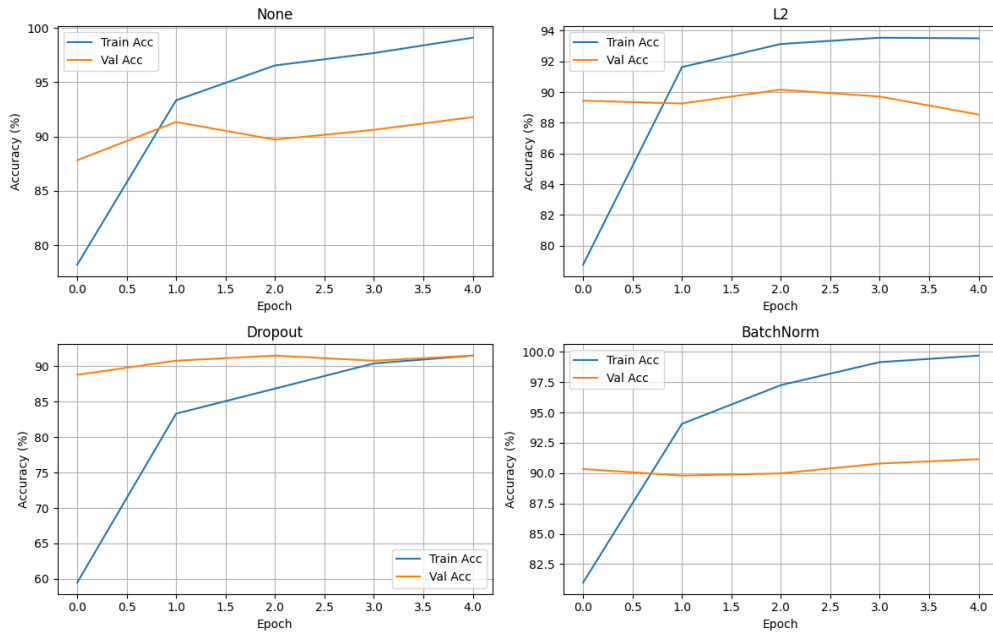
Plot 4: Training and Validation Loss vs. Epoch

- X-axis: Epoch
- Y-axis: Loss
- Plot training and validation loss.

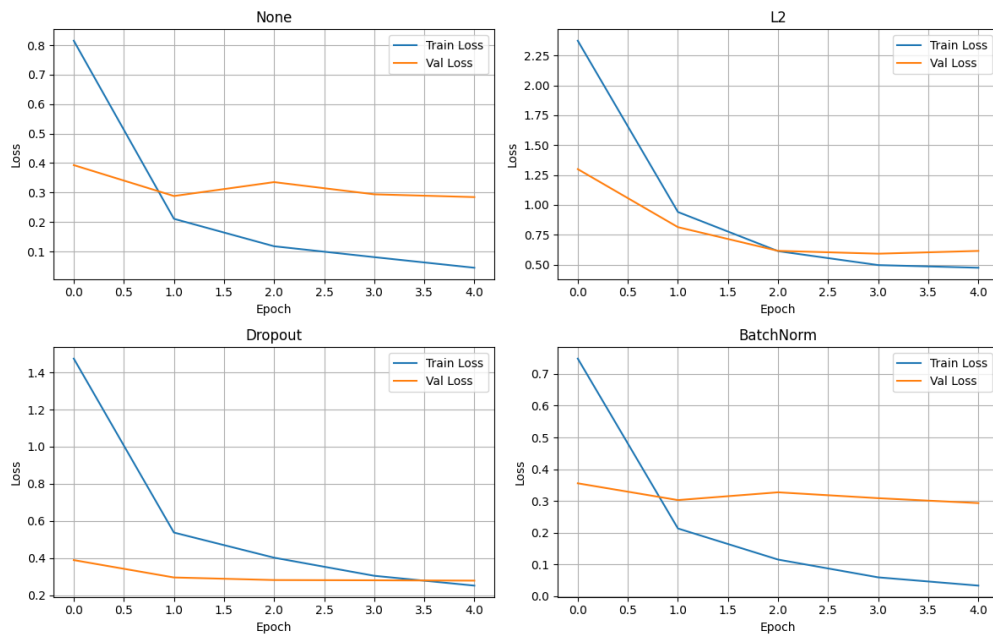
Students should identify whether validation loss begins to increase while training loss continues to decrease.

Output

Plot 3: Training vs Validation Accuracy



Plot 4: Training vs Validation Loss



--- Training with None regularization ---

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet_v2/mobilenet_v2_weights_9406464/9406464 0s 0us/step

Epoch 1/5

162/162 64s 268ms/step - accuracy: 0.7819 - loss: 0.8152 - val_accuracy: 0.8782 - val_loss: 0.3930

Epoch 2/5

162/162 15s 94ms/step - accuracy: 0.9333 - loss: 0.2108 - val_accuracy: 0.9134 - val_loss: 0.2881

Epoch 3/5

162/162 16s 97ms/step - accuracy: 0.9654 - loss: 0.1176 - val_accuracy: 0.8971 - val_loss: 0.3353

Epoch 4/5

162/162 20s 93ms/step - accuracy: 0.9768 - loss: 0.0807 - val_accuracy: 0.9061 - val_loss: 0.2939

Epoch 5/5

162/162 15s 92ms/step - accuracy: 0.9909 - loss: 0.0446 - val_accuracy: 0.9179 - val_loss: 0.2845

--- Training with L2 regularization ---

Epoch 1/5

162/162 37s 172ms/step - accuracy: 0.7876 - loss: 2.3743 - val_accuracy: 0.8944 - val_loss: 1.2995

Epoch 2/5

162/162 15s 92ms/step - accuracy: 0.9163 - loss: 0.9409 - val_accuracy: 0.8926 - val_loss: 0.8141

```

Epoch 3/5
162/162 20s 91ms/step - accuracy: 0.9314 - loss: 0.6132 - val_accuracy: 0.9016 - val_loss: 0.6156
Epoch 4/5
162/162 15s 91ms/step - accuracy: 0.9354 - loss: 0.4964 - val_accuracy: 0.8971 - val_loss: 0.5918
Epoch 5/5
162/162 15s 92ms/step - accuracy: 0.9350 - loss: 0.4742 - val_accuracy: 0.8854 - val_loss: 0.6151

--- Training with Dropout regularization ---
Epoch 1/5
162/162 37s 173ms/step - accuracy: 0.5946 - loss: 1.4740 - val_accuracy: 0.8881 - val_loss: 0.3900
Epoch 2/5
162/162 15s 92ms/step - accuracy: 0.8332 - loss: 0.5382 - val_accuracy: 0.9079 - val_loss: 0.2966
Epoch 3/5
162/162 21s 93ms/step - accuracy: 0.8685 - loss: 0.4030 - val_accuracy: 0.9152 - val_loss: 0.2827
Epoch 4/5
162/162 16s 96ms/step - accuracy: 0.9041 - loss: 0.3056 - val_accuracy: 0.9079 - val_loss: 0.2814
Epoch 5/5
162/162 17s 103ms/step - accuracy: 0.9151 - loss: 0.2528 - val_accuracy: 0.9152 - val_loss: 0.2796

--- Training with BatchNorm regularization ---
Epoch 1/5
162/162 37s 169ms/step - accuracy: 0.8098 - loss: 0.7482 - val_accuracy: 0.9034 - val_loss: 0.3556
Epoch 2/5
162/162 15s 92ms/step - accuracy: 0.9407 - loss: 0.2133 - val_accuracy: 0.8980 - val_loss: 0.3028
Epoch 3/5
162/162 15s 91ms/step - accuracy: 0.9725 - loss: 0.1154 - val_accuracy: 0.8998 - val_loss: 0.3273
Epoch 4/5
162/162 15s 93ms/step - accuracy: 0.9915 - loss: 0.0595 - val_accuracy: 0.9079 - val_loss: 0.3087
Epoch 5/5
162/162 20s 92ms/step - accuracy: 0.9969 - loss: 0.0335 - val_accuracy: 0.9116 - val_loss: 0.2933

```

7. Batch Normalization

Batch Normalization normalizes activations within a mini-batch during training.

For a mini-batch

$$x_1, x_2, \dots, x_m,$$

the batch mean is

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$$

and the batch variance is

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2.$$

The normalized activation is

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}.$$

The final output is

$$y_i = \gamma \hat{x}_i + \beta,$$

where γ and β are learnable parameters.

Numerical Example

Consider

$$x = [2, 4, 6, 8].$$

The mean is

$$\mu_B = \frac{2 + 4 + 6 + 8}{4} = 5.$$

The variance is

$$\sigma_B^2 = \frac{(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2}{4} = 5.$$

Therefore,

$$\sqrt{5} \approx 2.236.$$

Ignoring the very small ϵ ,

$$\hat{x}_1 = \frac{2-5}{2.236} \approx -1.342,$$

$$\hat{x}_2 = \frac{4-5}{2.236} \approx -0.447,$$

$$\hat{x}_3 = \frac{6-5}{2.236} \approx 0.447,$$

$$\hat{x}_4 = \frac{8-5}{2.236} \approx 1.342.$$

Hence,

$$\hat{x} \approx [-1.342, -0.447, 0.447, 1.342]$$

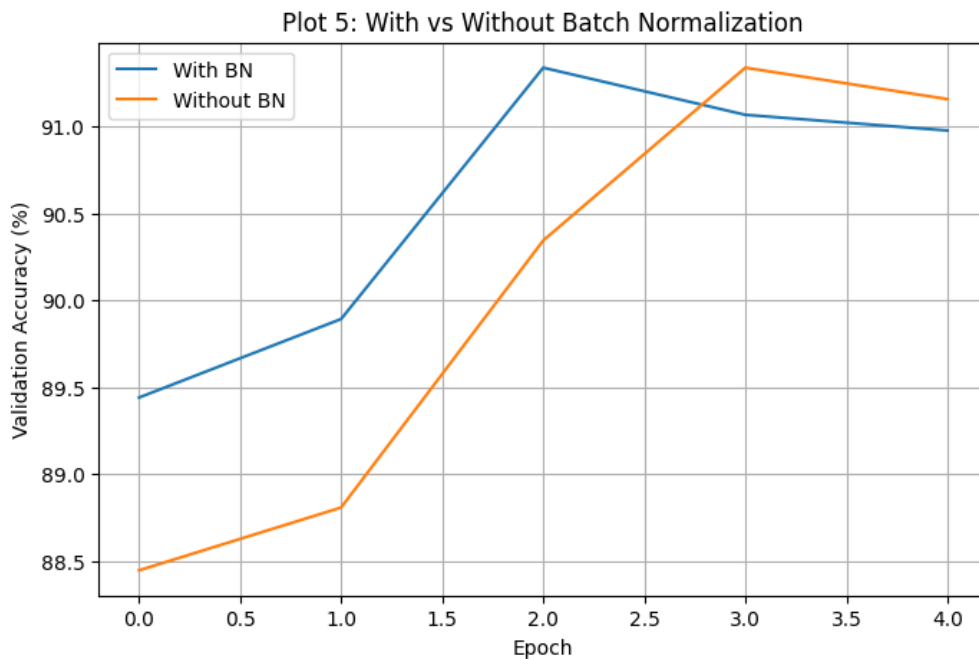
If $\gamma = 1$ and $\beta = 0$, these are the final outputs.

Inference: Batch Normalization produces approximately zero-centred and unit-scaled activations for this mini-batch. The learnable γ and β allow the network to learn an appropriate scale and shift.

Plot 5: With vs. Without Batch Normalization

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Compare two curves: With BN and Without BN.

Output



Batch mean (μ_B): 5.0
Batch variance (σ_B^2): 5.0
Normalized activations ($\hat{x}$): [-1.3416407 -0.4472136 0.4472136 1.3416407]
Final BN output ($\gamma=1, \beta=0$): [-1.3416407 -0.4472136 0.4472136 1.3416407]

--- Training With BN ---

Epoch 1/5
162/162 38s 175ms/step - accuracy: 0.7914 - loss: 0.9429 - val_accuracy: 0.8944 - val_loss: 0.3977
Epoch 2/5
162/162 15s 93ms/step - accuracy: 0.9412 - loss: 0.2624 - val_accuracy: 0.8989 - val_loss: 0.3299
Epoch 3/5
162/162 15s 93ms/step - accuracy: 0.9745 - loss: 0.1341 - val_accuracy: 0.9134 - val_loss: 0.2924
Epoch 4/5
162/162 15s 93ms/step - accuracy: 0.9865 - loss: 0.0741 - val_accuracy: 0.9106 - val_loss: 0.3043
Epoch 5/5
162/162 15s 93ms/step - accuracy: 0.9950 - loss: 0.0433 - val_accuracy: 0.9097 - val_loss: 0.2935

--- Training Without BN ---

Epoch 1/5
162/162 36s 172ms/step - accuracy: 0.7827 - loss: 0.8217 - val_accuracy: 0.8845 - val_loss: 0.3483
Epoch 2/5
162/162 15s 95ms/step - accuracy: 0.9318 - loss: 0.2129 - val_accuracy: 0.8881 - val_loss: 0.3334
Epoch 3/5
162/162 23s 110ms/step - accuracy: 0.9623 - loss: 0.1193 - val_accuracy: 0.9034 - val_loss: 0.2831
Epoch 4/5
162/162 15s 93ms/step - accuracy: 0.9838 - loss: 0.0698 - val_accuracy: 0.9134 - val_loss: 0.2700
Epoch 5/5
162/162 15s 94ms/step - accuracy: 0.9911 - loss: 0.0448 - val_accuracy: 0.9116 - val_loss: 0.3072

8. Optimization Algorithms

Students should compare the following optimization methods:

- SGD
- Momentum
- RMSProp
- Adam

The objective is to study convergence speed, stability and final validation performance.

Plot 6: Training Loss vs. Epoch for Different Optimizers

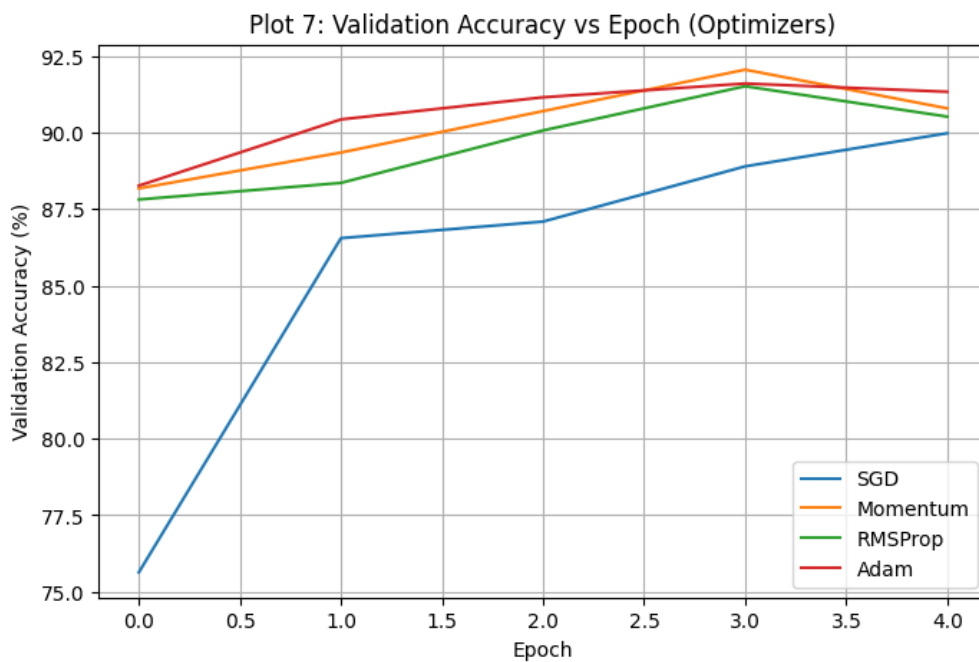
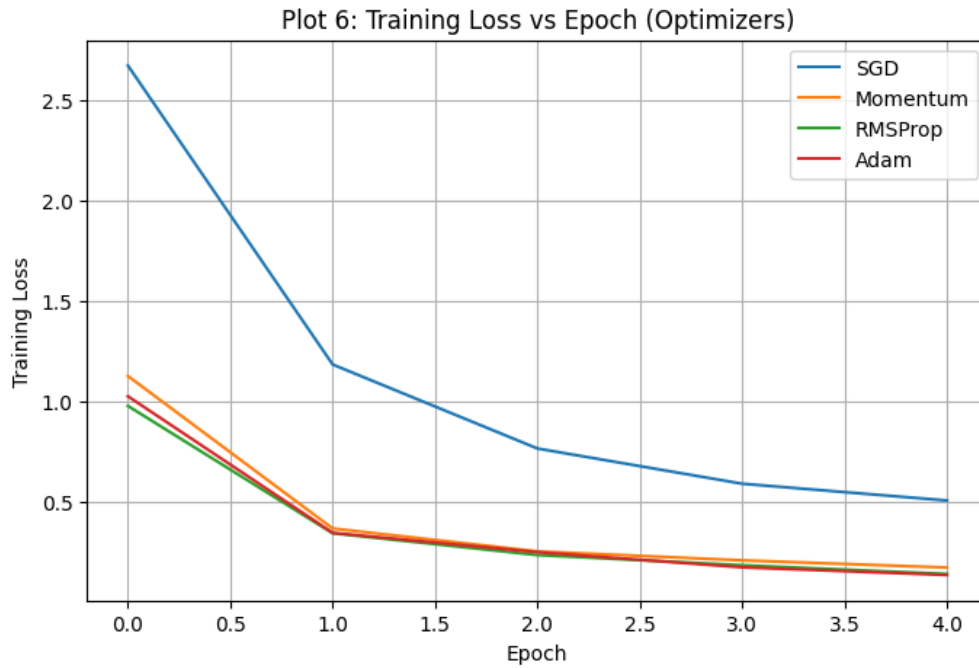
- X-axis: Epoch
- Y-axis: Training Loss
- One curve for each optimizer.

Plot 7: Validation Accuracy vs. Epoch for Different Optimizers

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- One curve for each optimizer.

Optimizer	Final Loss	Best Val. Accuracy	Epoch to Converge	Time
SGD	0.5063	89.98%	5	114.5s
Momentum	0.1726	92.06%	4	106.5s
RMSProp	0.1406	91.52%	4	108.5s
Adam	0.1355	91.61%	4	109.6s

Output



--- Training with SGD ---

Epoch 1/5

162/162 37s 174ms/step - accuracy: 0.3396 - loss: 2.6707 - val_accuracy: 0.7563 - val_loss: 1.4407

Epoch 2/5

162/162 24s 97ms/step - accuracy: 0.7205 - loss: 1.1831 - val_accuracy: 0.8655 - val_loss: 0.6778

Epoch 3/5

162/162 18s 109ms/step - accuracy: 0.7955 - loss: 0.7651 - val_accuracy: 0.8709 - val_loss: 0.5000

Epoch 4/5

162/162 15s 95ms/step - accuracy: 0.8399 - loss: 0.5897 - val_accuracy: 0.8890 - val_loss: 0.4174

Epoch 5/5

162/162 15s 92ms/step - accuracy: 0.8556 - loss: 0.5063 - val_accuracy: 0.8998 - val_loss: 0.3683

--- Training with Momentum ---

Epoch 1/5

162/162 38s 179ms/step - accuracy: 0.6886 - loss: 1.1256 - val_accuracy: 0.8818 - val_loss: 0.3638

Epoch 2/5

162/162 15s 93ms/step - accuracy: 0.8830 - loss: 0.3669 - val_accuracy: 0.8935 - val_loss: 0.3380

Epoch 3/5

```

162/162 21s 93ms/step - accuracy: 0.9221 - loss: 0.2530 - val_accuracy: 0.9070 - val_loss: 0.2915
Epoch 4/5
162/162 18s 110ms/step - accuracy: 0.9341 - loss: 0.2084 - val_accuracy: 0.9206 - val_loss: 0.2652
Epoch 5/5
162/162 15s 94ms/step - accuracy: 0.9436 - loss: 0.1726 - val_accuracy: 0.9079 - val_loss: 0.2752

```

--- Training with RMSProp ---

```

Epoch 1/5
162/162 38s 174ms/step - accuracy: 0.7274 - loss: 0.9768 - val_accuracy: 0.8782 - val_loss: 0.3846
Epoch 2/5
162/162 15s 94ms/step - accuracy: 0.8871 - loss: 0.3431 - val_accuracy: 0.8836 - val_loss: 0.3661
Epoch 3/5
162/162 23s 107ms/step - accuracy: 0.9254 - loss: 0.2347 - val_accuracy: 0.9007 - val_loss: 0.2995
Epoch 4/5
162/162 18s 93ms/step - accuracy: 0.9362 - loss: 0.1839 - val_accuracy: 0.9152 - val_loss: 0.2768
Epoch 5/5
162/162 15s 92ms/step - accuracy: 0.9509 - loss: 0.1406 - val_accuracy: 0.9052 - val_loss: 0.3136

```

--- Training with Adam ---

```

Epoch 1/5
162/162 38s 174ms/step - accuracy: 0.7265 - loss: 1.0248 - val_accuracy: 0.8827 - val_loss: 0.3732
Epoch 2/5
162/162 15s 93ms/step - accuracy: 0.8950 - loss: 0.3454 - val_accuracy: 0.9043 - val_loss: 0.2986
Epoch 3/5
162/162 20s 93ms/step - accuracy: 0.9229 - loss: 0.2491 - val_accuracy: 0.9116 - val_loss: 0.2819
Epoch 4/5
162/162 21s 97ms/step - accuracy: 0.9428 - loss: 0.1736 - val_accuracy: 0.9161 - val_loss: 0.2710
Epoch 5/5
162/162 15s 94ms/step - accuracy: 0.9555 - loss: 0.1355 - val_accuracy: 0.9134 - val_loss: 0.2767

```

Optimizer	Final Loss	Best Val. Accuracy	Epoch to Converge	Time (s)
SGD	0.5063	89.98	5	114.5
Momentum	0.1726	92.06	4	106.5
RMSProp	0.1406	91.52	4	108.5
Adam	0.1355	91.61	4	109.6

9. CNN Hyperparameter Tuning

The following hyperparameters should be investigated:

Hyperparameter	Suggested Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Dropout Rate	0, 0.25, 0.5
Optimizer	SGD, Adam
Fine-Tuning Learning Rate	10^{-4} , 10^{-5}
Frozen Layers	Frozen base / Partial unfreezing

For CNN concepts, students should also understand:

- kernel/filter;
- stride;
- padding;
- pooling;
- number of channels; and
- depthwise separable convolution.

For a convolution operation, the output dimension can be calculated as

$$O = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1,$$

where N is input size, K is kernel size, P is padding and S is stride.

Experimental rule: During a controlled hyperparameter study, change one hyperparameter at a time while keeping the remaining settings fixed.

Plot 8: Learning Rate vs. Validation Accuracy

- X-axis: Learning Rate
- Y-axis: Validation Accuracy (%)

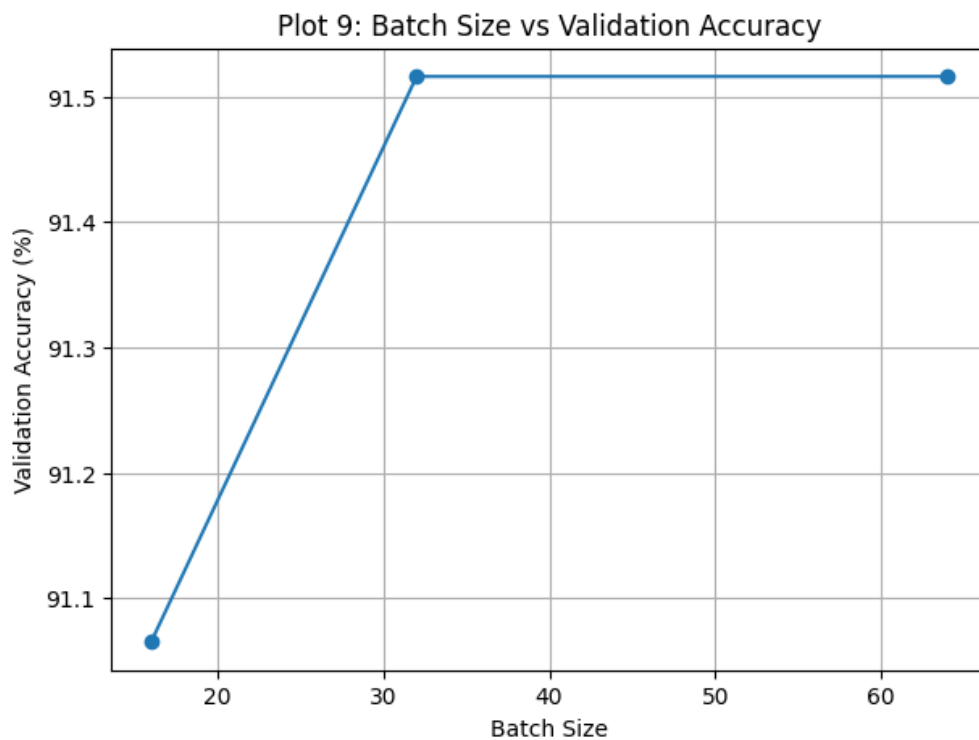
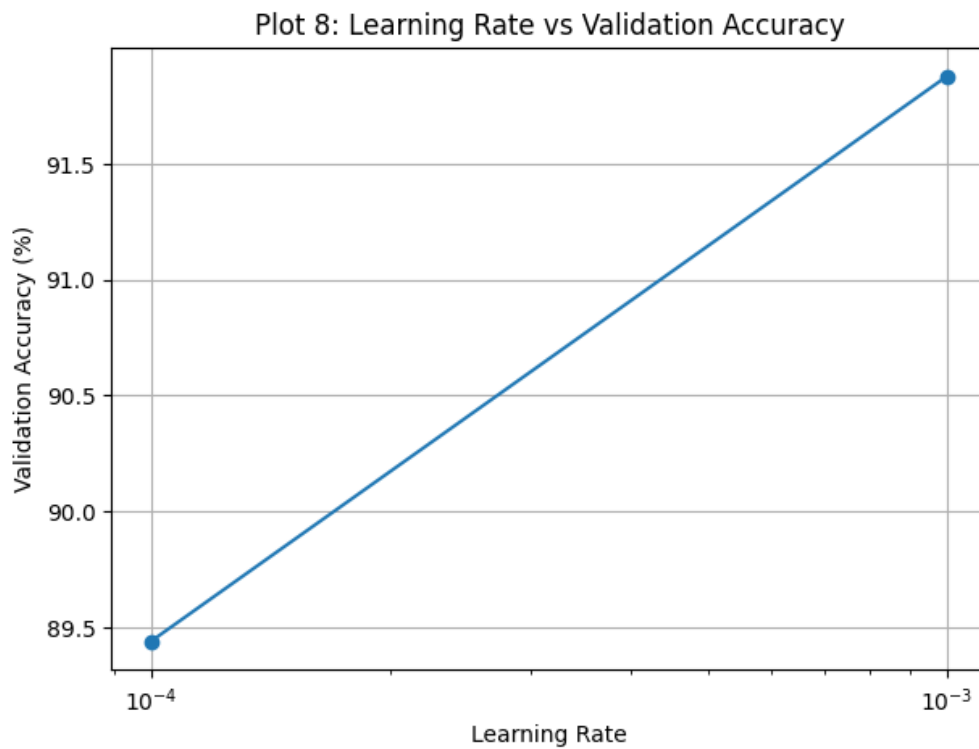
Plot 9: Batch Size vs. Validation Accuracy

- X-axis: Batch Size
- Y-axis: Validation Accuracy (%)

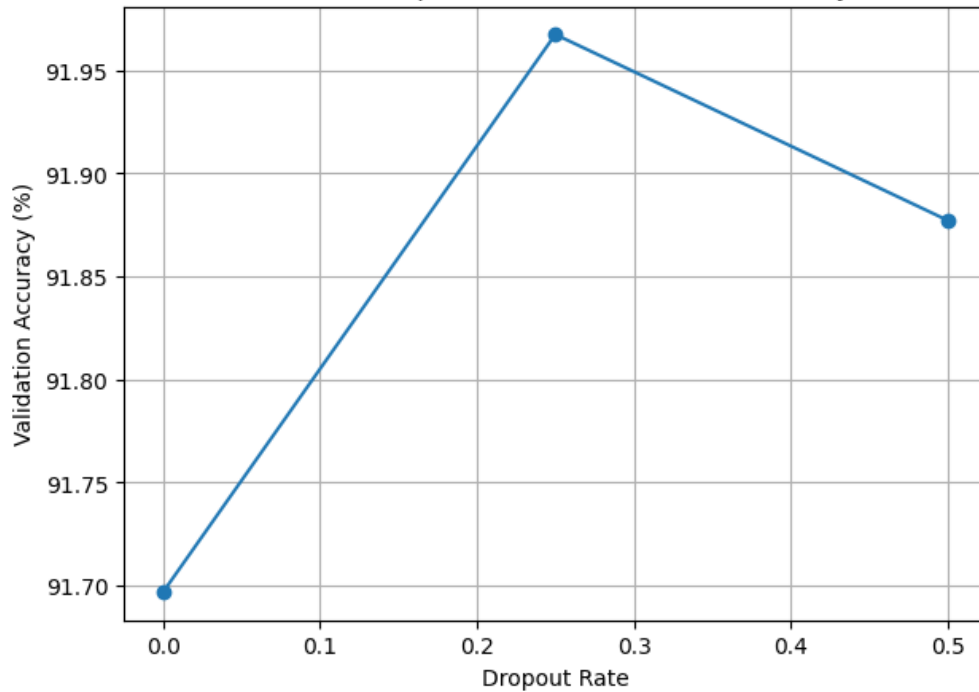
Plot 10: Dropout Rate vs. Validation Accuracy

- X-axis: Dropout Rate
- Y-axis: Validation Accuracy (%)

Output



Plot 10: Dropout Rate vs Validation Accuracy



--- Learning Rate: 0.001 ---

Epoch 1/5
 162/162 39s 176ms/step - accuracy: 0.7079 - loss: 1.0755 - val_accuracy: 0.8917 - val_loss: 0.3431
 Epoch 2/5
 162/162 15s 94ms/step - accuracy: 0.8927 - loss: 0.3433 - val_accuracy: 0.9152 - val_loss: 0.2705
 Epoch 3/5
 162/162 20s 94ms/step - accuracy: 0.9217 - loss: 0.2544 - val_accuracy: 0.9088 - val_loss: 0.2810
 Epoch 4/5
 162/162 15s 92ms/step - accuracy: 0.9441 - loss: 0.1740 - val_accuracy: 0.9179 - val_loss: 0.2636
 Epoch 5/5
 162/162 23s 110ms/step - accuracy: 0.9582 - loss: 0.1323 - val_accuracy: 0.9188 - val_loss: 0.2624

--- Learning Rate: 0.0001 ---

Epoch 1/5
 162/162 38s 177ms/step - accuracy: 0.2770 - loss: 2.8875 - val_accuracy: 0.6850 - val_loss: 1.7681
 Epoch 2/5
 162/162 15s 94ms/step - accuracy: 0.6752 - loss: 1.3830 - val_accuracy: 0.8493 - val_loss: 0.7977
 Epoch 3/5
 162/162 15s 94ms/step - accuracy: 0.7953 - loss: 0.8138 - val_accuracy: 0.8736 - val_loss: 0.5493
 Epoch 4/5
 162/162 18s 111ms/step - accuracy: 0.8355 - loss: 0.6221 - val_accuracy: 0.8944 - val_loss: 0.4459
 Epoch 5/5
 162/162 18s 93ms/step - accuracy: 0.8593 - loss: 0.5145 - val_accuracy: 0.8926 - val_loss: 0.3995

--- Batch Size: 16 ---

Epoch 1/5
 324/324 65s 146ms/step - accuracy: 0.7404 - loss: 0.9506 - val_accuracy: 0.8980 - val_loss: 0.3450
 Epoch 2/5
 324/324 18s 57ms/step - accuracy: 0.8981 - loss: 0.3310 - val_accuracy: 0.8926 - val_loss: 0.3333
 Epoch 3/5
 324/324 16s 49ms/step - accuracy: 0.9277 - loss: 0.2245 - val_accuracy: 0.9007 - val_loss: 0.3218
 Epoch 4/5
 324/324 21s 50ms/step - accuracy: 0.9408 - loss: 0.1742 - val_accuracy: 0.9106 - val_loss: 0.2932
 Epoch 5/5
 324/324 20s 49ms/step - accuracy: 0.9534 - loss: 0.1417 - val_accuracy: 0.8971 - val_loss: 0.3171

--- Batch Size: 32 ---

Epoch 1/5
 162/162 39s 175ms/step - accuracy: 0.7180 - loss: 1.0078 - val_accuracy: 0.8863 - val_loss: 0.3658
 Epoch 2/5
 162/162 15s 94ms/step - accuracy: 0.8910 - loss: 0.3410 - val_accuracy: 0.8980 - val_loss: 0.3170
 Epoch 3/5

```

162/162 21s 95ms/step - accuracy: 0.9258 - loss: 0.2346 - val_accuracy: 0.9152 - val_loss: 0.2908
Epoch 4/5
162/162 15s 94ms/step - accuracy: 0.9436 - loss: 0.1695 - val_accuracy: 0.9152 - val_loss: 0.2727
Epoch 5/5
162/162 15s 93ms/step - accuracy: 0.9550 - loss: 0.1380 - val_accuracy: 0.9043 - val_loss: 0.2914

--- Batch Size: 64 ---
Epoch 1/5
81/81 58s 464ms/step - accuracy: 0.6627 - loss: 1.2649 - val_accuracy: 0.8745 - val_loss: 0.4034
Epoch 2/5
81/81 14s 176ms/step - accuracy: 0.8830 - loss: 0.3793 - val_accuracy: 0.8971 - val_loss: 0.3197
Epoch 3/5
81/81 14s 178ms/step - accuracy: 0.9198 - loss: 0.2661 - val_accuracy: 0.9152 - val_loss: 0.2977
Epoch 4/5
81/81 21s 183ms/step - accuracy: 0.9339 - loss: 0.2027 - val_accuracy: 0.9152 - val_loss: 0.2819
Epoch 5/5
81/81 14s 178ms/step - accuracy: 0.9445 - loss: 0.1733 - val_accuracy: 0.9016 - val_loss: 0.2995

--- Dropout Rate: 0 ---
Epoch 1/5
162/162 37s 172ms/step - accuracy: 0.7798 - loss: 0.8191 - val_accuracy: 0.8899 - val_loss: 0.3511
Epoch 2/5
162/162 15s 95ms/step - accuracy: 0.9341 - loss: 0.2037 - val_accuracy: 0.9097 - val_loss: 0.3042
Epoch 3/5
162/162 16s 97ms/step - accuracy: 0.9579 - loss: 0.1278 - val_accuracy: 0.8953 - val_loss: 0.3271
Epoch 4/5
162/162 20s 94ms/step - accuracy: 0.9832 - loss: 0.0722 - val_accuracy: 0.9170 - val_loss: 0.3006
Epoch 5/5
162/162 23s 108ms/step - accuracy: 0.9919 - loss: 0.0441 - val_accuracy: 0.9025 - val_loss: 0.3097

--- Dropout Rate: 0.25 ---
Epoch 1/5
162/162 38s 174ms/step - accuracy: 0.7253 - loss: 1.0044 - val_accuracy: 0.9106 - val_loss: 0.3164
Epoch 2/5
162/162 15s 93ms/step - accuracy: 0.8919 - loss: 0.3305 - val_accuracy: 0.9016 - val_loss: 0.2936
Epoch 3/5
162/162 15s 94ms/step - accuracy: 0.9223 - loss: 0.2306 - val_accuracy: 0.9197 - val_loss: 0.2833
Epoch 4/5
162/162 20s 92ms/step - accuracy: 0.9472 - loss: 0.1589 - val_accuracy: 0.9152 - val_loss: 0.2605
Epoch 5/5
162/162 15s 93ms/step - accuracy: 0.9600 - loss: 0.1286 - val_accuracy: 0.9088 - val_loss: 0.2812

--- Dropout Rate: 0.5 ---
Epoch 1/5
162/162 39s 175ms/step - accuracy: 0.6122 - loss: 1.4364 - val_accuracy: 0.8845 - val_loss: 0.4066
Epoch 2/5
162/162 16s 97ms/step - accuracy: 0.8367 - loss: 0.5370 - val_accuracy: 0.9088 - val_loss: 0.3083
Epoch 3/5
162/162 20s 94ms/step - accuracy: 0.8800 - loss: 0.3784 - val_accuracy: 0.9188 - val_loss: 0.2802
Epoch 4/5
162/162 23s 108ms/step - accuracy: 0.9082 - loss: 0.2967 - val_accuracy: 0.9016 - val_loss: 0.3014
Epoch 5/5
162/162 15s 94ms/step - accuracy: 0.9153 - loss: 0.2597 - val_accuracy: 0.9143 - val_loss: 0.2714

```

10. Transfer Learning and Fine-Tuning

MobileNetV2 pretrained on ImageNet should be used.

Case A: Feature Extraction

Pretrained MobileNetV2 → Freeze Base → New Classifier

The pretrained convolutional layers are frozen and only the newly added classification layers are trained.

Case B: Fine-Tuning

Pretrained MobileNetV2 → Unfreeze Selected Layers → Small Learning Rate

The upper layers of the pretrained network are unfrozen and trained with a smaller learning rate.

Plot 11: Feature Extraction vs. Fine-Tuning

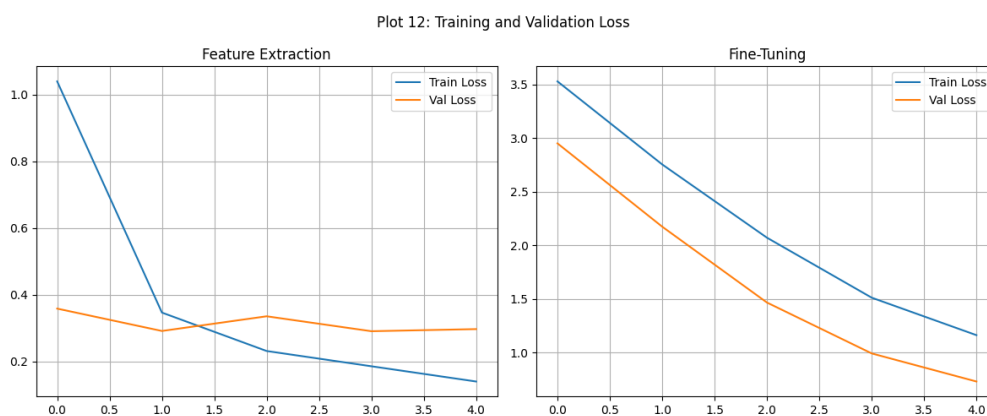
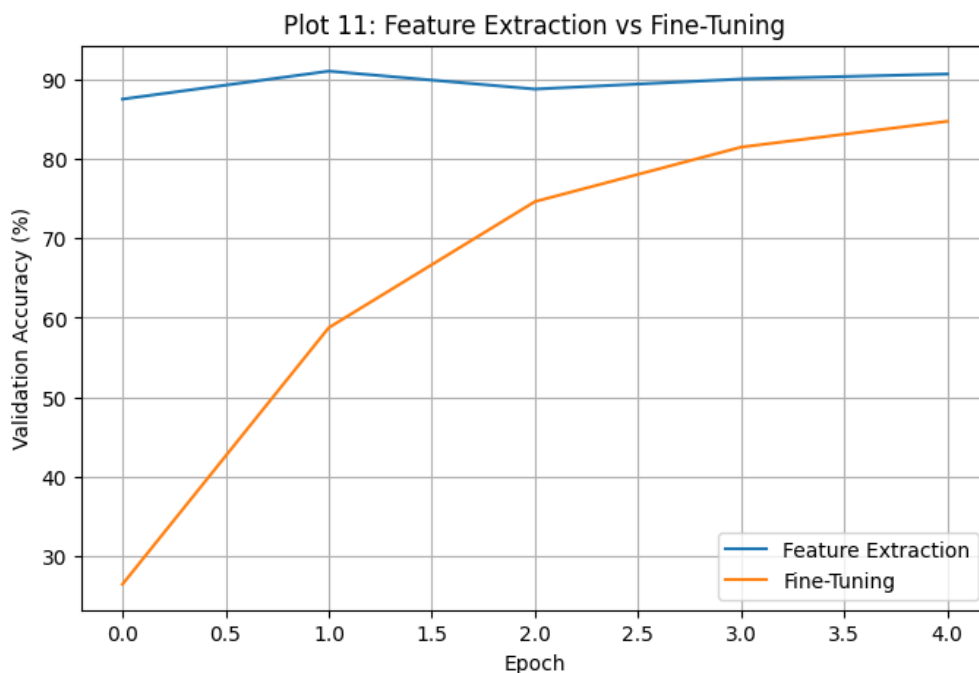
- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Two curves: Feature Extraction and Fine-Tuning.

Plot 12: Training and Validation Loss

Students should compare the loss curves before and after fine-tuning.

Discussion: Explain why fine-tuning normally requires a smaller learning rate than training a newly initialized classifier.

Output



--- Case A: Feature Extraction ---

Epoch 1/5

162/162 38s 174ms/step - accuracy: 0.7176 - loss: 1.0390 - val_accuracy: 0.8755 - val_loss: 0.3586

Epoch 2/5

162/162 15s 93ms/step - accuracy: 0.8875 - loss: 0.3467 - val_accuracy: 0.9106 - val_loss: 0.2916

Epoch 3/5

162/162 20s 92ms/step - accuracy: 0.9291 - loss: 0.2316 - val_accuracy: 0.8881 - val_loss: 0.3356

Epoch 4/5

162/162 21s 93ms/step - accuracy: 0.9403 - loss: 0.1859 - val_accuracy: 0.9007 - val_loss: 0.2909

Epoch 5/5

162/162 20s 93ms/step - accuracy: 0.9548 - loss: 0.1403 - val_accuracy: 0.9070 - val_loss: 0.2972

--- Case B: Fine-Tuning ---

Epoch 1/5

162/162 49s 201ms/step - accuracy: 0.0762 - loss: 3.5292 - val_accuracy: 0.2644 - val_loss: 2.9507

Epoch 2/5

162/162 16s 94ms/step - accuracy: 0.2983 - loss: 2.7542 - val_accuracy: 0.5875 - val_loss: 2.1734

Epoch 3/5

162/162 15s 93ms/step - accuracy: 0.5248 - loss: 2.0706 - val_accuracy: 0.7464 - val_loss: 1.4656

Epoch 4/5

162/162 15s 94ms/step - accuracy: 0.6625 - loss: 1.5109 - val_accuracy: 0.8150 - val_loss: 0.9907

Epoch 5/5

162/162 21s 96ms/step - accuracy: 0.7315 - loss: 1.1612 - val_accuracy: 0.8475 - val_loss: 0.7286

11. K-Fold Cross-Validation

After the individual studies, select 3–4 promising configurations.

Use 5-fold cross-validation on the training data.

For $K = 5$,

$$\bar{A} = \frac{1}{5} \sum_{i=1}^5 A_i$$

and

$$SD = \sqrt{\frac{1}{5} \sum_{i=1}^5 (A_i - \bar{A})^2}.$$

Report the result as

$$\boxed{\bar{A} \pm SD}.$$

The independent test set must not be used during hyperparameter selection.

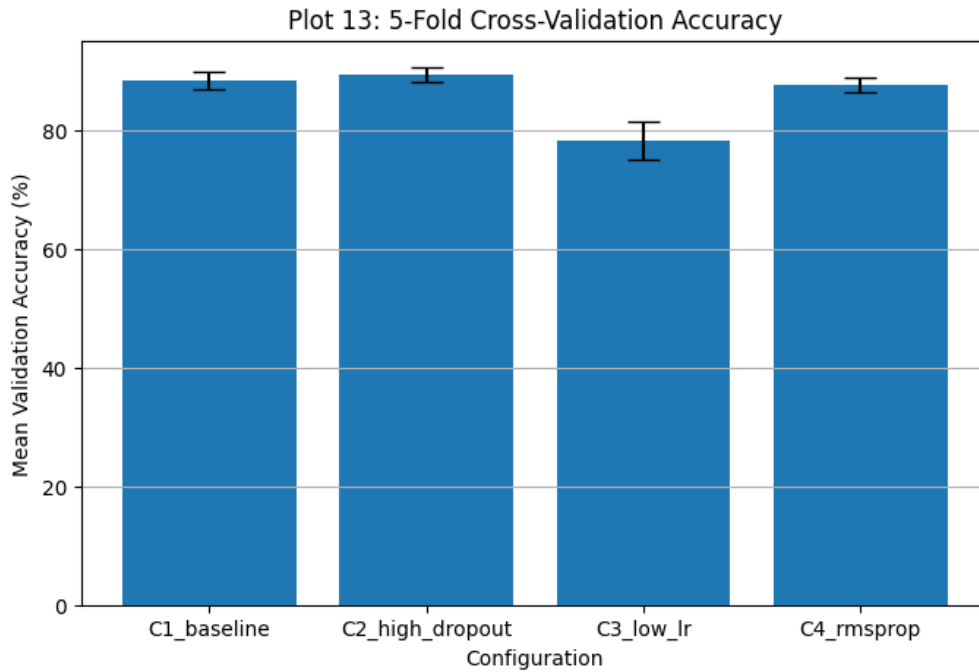
Configuration	F1	F2	F3	F4	F5	Mean $\pm$ SD
C1 (Baseline)	88.75%	86.50%	90.75%	87.50%	88.00%	88.30 ± 1.43
C2 (High Dropout)	89.50%	87.25%	90.25%	90.75%	89.25%	89.40 ± 1.20
C3 (Low LR)	76.75%	73.25%	82.25%	77.75%	81.25%	78.25 ± 3.24
C4 (RMSProp)	88.00%	86.00%	89.75%	87.50%	87.25%	87.70 ± 1.22

Plot 13: 5-Fold Cross-Validation Accuracy

- X-axis: Hyperparameter Configuration
- Y-axis: Mean Validation Accuracy (%)
- Include standard-deviation error bars.

Students should discuss both mean performance and variability.

Output



Total samples for CV: 6281
Subsampled to: 2000

=== Config: C1_baseline ===

Fold 1: Val Accuracy = 88.75%
Fold 2: Val Accuracy = 86.50%
Fold 3: Val Accuracy = 90.75%
Fold 4: Val Accuracy = 87.50%
Fold 5: Val Accuracy = 88.00%

=== Config: C2_high_dropout ===

Fold 1: Val Accuracy = 89.50%
Fold 2: Val Accuracy = 87.25%
Fold 3: Val Accuracy = 90.25%
Fold 4: Val Accuracy = 90.75%
Fold 5: Val Accuracy = 89.25%

=== Config: C3_low_lr ===

Fold 1: Val Accuracy = 76.75%
Fold 2: Val Accuracy = 73.25%
Fold 3: Val Accuracy = 82.25%
Fold 4: Val Accuracy = 77.75%
Fold 5: Val Accuracy = 81.25%

=== Config: C4_rmsprop ===

Fold 1: Val Accuracy = 88.00%
Fold 2: Val Accuracy = 86.00%
Fold 3: Val Accuracy = 89.75%
Fold 4: Val Accuracy = 87.50%
Fold 5: Val Accuracy = 87.25%

Configuration	F1	F2	F3	F4	F5	Mean	SD
C1_baseline	88.75	86.50	90.75	87.50	88.00	88.30	1.43
C2_high_dropout	89.50	87.25	90.25	90.75	89.25	89.40	1.20
C3_low_lr	76.75	73.25	82.25	77.75	81.25	78.25	3.24
C4_rmsprop	88.00	86.00	89.75	87.50	87.25	87.70	1.22

12. Final Model Evaluation

After selecting the best configuration using cross-validation:

1. Retrain the selected configuration using the complete training data.

2. Keep the independent test set untouched until this stage.
3. Evaluate the final model on the test set.

Report:

Metric	Value
Mean CV Accuracy	89.40%
CV Standard Deviation	1.20%
Test Accuracy	89.81%
Precision	90.47%
Recall	89.81%
F1-score	89.70%
Training Time	70.10s
Number of Parameters	2,426,725

Plot 14: Confusion Matrix

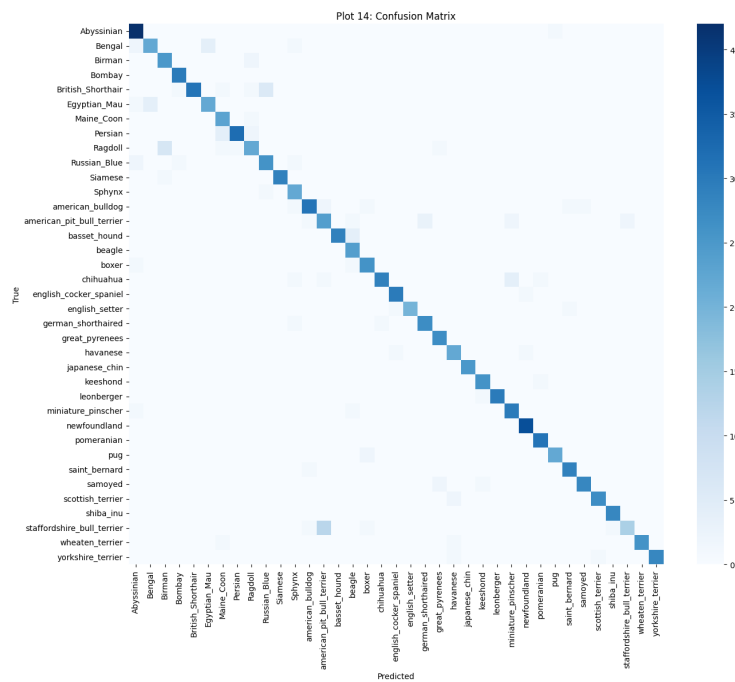
Students should identify:

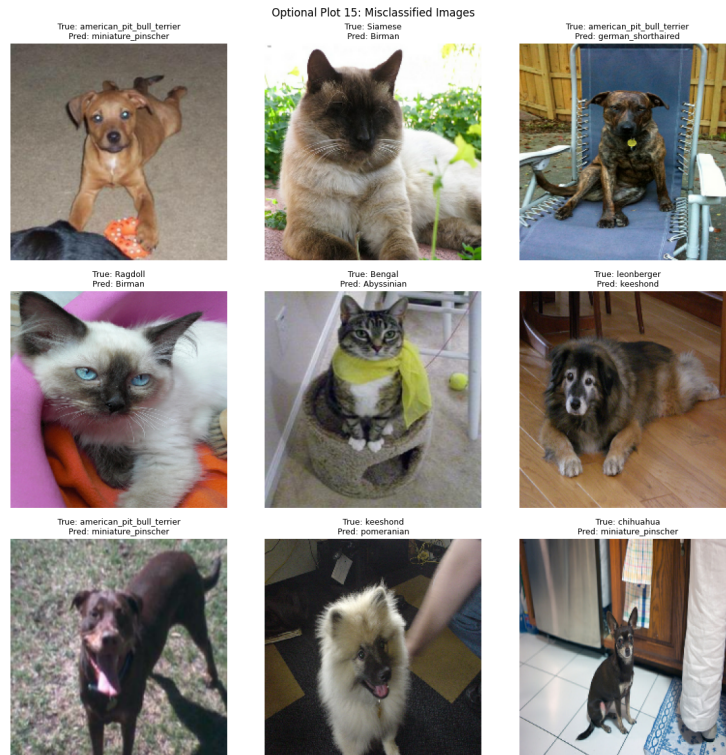
- the best classified classes;
- the most frequently confused classes; and
- possible visual reasons for misclassification.

Optional Plot 15: Misclassified Images

Display representative incorrectly classified images and provide a brief explanation for each.

Output





Best configuration selected: C2_high_dropout

Epoch 1/10
63/63 21s 153ms/step - accuracy: 0.4135 - loss: 2.2573
Epoch 2/10
63/63 4s 67ms/step - accuracy: 0.7530 - loss: 0.8332
Epoch 3/10
63/63 6s 90ms/step - accuracy: 0.8420 - loss: 0.5436
Epoch 4/10
63/63 9s 67ms/step - accuracy: 0.8785 - loss: 0.3763
Epoch 5/10
63/63 7s 90ms/step - accuracy: 0.9055 - loss: 0.3097
Epoch 6/10
63/63 4s 68ms/step - accuracy: 0.9335 - loss: 0.2363
Epoch 7/10
63/63 5s 76ms/step - accuracy: 0.9375 - loss: 0.2121
Epoch 8/10
63/63 5s 79ms/step - accuracy: 0.9455 - loss: 0.1852
Epoch 9/10
63/63 4s 68ms/step - accuracy: 0.9590 - loss: 0.1499
Epoch 10/10
63/63 6s 88ms/step - accuracy: 0.9610 - loss: 0.1341
Test Accuracy: 89.81%

	0
Mean CV Accuracy	89.40
CV Standard Deviation	1.20
Test Accuracy	89.81
Precision	90.47
Recall	89.81
F1-score	89.70
Training Time (s)	70.10
Number of Parameters	2426725.00
0	

13. Overall Results

Configuration	CV Accuracy	SD	Test Accuracy	Training Time
Baseline (No Regularization)	91.79%*	–	–	130.0s
Best Initialization (He)	91.43%*	–	–	97.0s
Best Regularization (Dropout)	91.52%*	–	–	106.0s
Best Optimizer (Momentum)	92.06%*	–	–	107.0s
Best Hyperparameters (Dropout 0.25)	91.97%*	–	–	103.0s
Fine-Tuned Model	89.40%	1.20%	89.81%	70.1s

*These rows report the best single-split validation accuracy achieved in their respective task (Tasks 1, 2, 4 and 5), since only the final selected configuration (Task 8) was evaluated with 5-fold cross-validation and on the held-out test set.

Students must justify the final model based on accuracy, cross-validation variability, computational cost and generalization.

14. Required Inference for Plots

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?

Each inference should be approximately 2–3 lines.

15. Discussion Questions

1. What is the difference between model parameters and hyperparameters?
2. Why is weight initialization important?
3. Why can zero initialization be problematic for neural networks?
4. Compare Xavier and He initialization.
5. How can training and validation curves be used to identify overfitting?
6. How does Dropout reduce overfitting?
7. What is the purpose of Batch Normalization?
8. Explain the numerical Batch Normalization example.
9. What are the roles of γ and β ?
10. Compare SGD, Momentum, RMSProp and Adam.
11. What happens when the learning rate is too large?
12. What happens when the learning rate is too small?
13. What is the effect of increasing batch size?
14. Explain stride and padding.
15. Why is MobileNetV2 computationally efficient?
16. What is depthwise separable convolution?
17. What is transfer learning?
18. Differentiate feature extraction and fine-tuning.
19. Why is a smaller learning rate generally used during fine-tuning?
20. Why is K-Fold Cross-Validation useful for hyperparameter selection?
21. Why must the test set remain untouched during tuning?
22. Why should mean and standard deviation both be reported?
23. Is the highest validation accuracy always sufficient to select a model?

16. Additional Exercise

Select two new combinations of learning rate, dropout, batch size and fine-tuning strategy. Evaluate them using 5-fold cross-validation and compare their results with the selected configuration.

Students must justify whether the new configuration is preferable based on accuracy, standard deviation, computational cost and final test performance.

17. Expected Outcome

Students should experimentally demonstrate that CNN performance is strongly influenced by initialization, regularization, optimization and hyperparameter choices. They should understand the MobileNetV2 architecture, perform transfer learning and fine-tuning, and select a reliable final configuration using 5-fold cross-validation.

18. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sergey Ioffe and Christian Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” ICML, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” CVPR, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, “Cats and Dogs,” IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>